

tastemam Catering Web Application

Project Report

CENG 382 — Web Development

Ata Soytürk

Student ID: 202211063

Date: May 2026

1. INTRODUCTION

This report explains in detail the tastemam Catering web application developed within the scope of the CENG 382 Web Development course. Tastemam is a comprehensive web platform with role-based access control that connects catering companies operating in 25 districts of Ankara with customers.

The platform allows users to discover nearby catering companies, select and customize menus, place customized orders, perform payment simulations, communicate with caretakers through live meetings, and have all system events logged systematically in a modern web application environment.

2. PROJECT PURPOSE AND SCOPE

The main objective of the project is to develop a secure, scalable, and user-friendly catering platform that reflects real-world scenarios.

The platform provides the following core functionalities:

- Role-based authentication and authorization
- Dynamic menu management and customization system
- Location-based filtering
- Order and payment simulation
- Dynamic PDF document generation
- Real-time video meetings
- AI-powered customer assistant
- Comprehensive system logging

3. TECHNOLOGIES USED

3.1 Backend

- ASP.NET Core MVC (.NET 10) — Web application framework
- Entity Framework Core — ORM and Code First database management
- ASP.NET Core Identity — Authentication and authorization
- Microsoft SQL Server (LocalDB) — Database management system

3.2 Frontend

- Bootstrap 5.3 — Responsive design
- Font Awesome 6.5 — Icon library
- Google Fonts (Playfair Display, Lato) — Typography
- Vanilla JavaScript — Client-side interactions

3.3 External Services and Libraries

- Google Maps JavaScript API — Location-based filtering
- Jitsi Meet External API — Live video conferencing
- Google Gemini API — AI chatbot integration
- MailKit — Email sending
- QuestPDF — Dynamic PDF generation
- X.PagedList — Pagination
- OWASP ZAP — Security analysis

4. SYSTEM ARCHITECTURE

4.1 MVC Architecture

The application adopts the (MVC) architecture. Thanks to this architecture, business logic, presentation layer, and data layer are separated from each other.

Controllers:

- HomeController — Homepage, menu listing, menu detail, user dashboard, and order history
- AccountController — Registration, login, logout, and two-factor authentication
- CaretakerController — Menu CRUD, customization, ingredient, and agreement management
- OrderController — Checkout, payment, order completion, receipt download, and meeting
- CartController — Cart operations
- RatingController — Menu and caretaker ratings
- AdminController — Dashboard, user, caretaker, order, log, and rating management

- ChatbotController — Gemini API integration

Services:

- LogService — System logging
- EmailService — Email sending (MailKit)
- PdfService — Dynamic PDF generation (QuestPDF)
- ChatbotService — Gemini API communication

4.2 Layered Structure

Controllers → Services → Data (AppDbContext) → Models

5. DATABASE DESIGN

5.1 Database Approach

The Code First approach with Entity Framework Core was adopted for database management. Database schemas were automatically generated through migrations based on C# model classes.

5.2 Tables and Relationships

AspNetUsers (ASP.NET Identity)

Stores user credentials, password hashes, emails, and role assignments.

AspNetRoles / AspNetUserRoles

Stores Admin, Caretaker, and User roles.

MenuItems

Contains menu information including name, description, price, category, image path, caretaker ID, location coordinates, and minimum order quantity.

CustomizationGroups

Stores menu customization groups.

CustomizationOptions

Stores selectable customization options.

Ingredients

Stores ingredient lists and removable ingredient information.

Orders

Stores customer information, order date, total price, state, and user ID.

OrderItems

Stores ordered menu items and quantities.

OrderItemCustomizations

Stores selected customization options for order items.

Ratings

Stores menu and caretaker ratings.

SystemLogs

Stores system activity logs.

CaretakerAgreements

Stores caretaker agreement records.

5.3 Relationships

- Menu → CustomizationGroups: One-to-Many
- CustomizationGroup → CustomizationOptions: One-to-Many
- Menu → Ingredients: One-to-Many
- Order → OrderItems: One-to-Many
- OrderItem → OrderItemCustomizations: One-to-Many
- Order → Ratings: One-to-Many

6. IMPLEMENTED FEATURES

6.1 Authentication and Authorization

ASP.NET Core Identity is used for secure authentication and authorization. Three roles are defined: Admin, Caretaker, and User.

Two-Factor Authentication (2FA):

Email-based two-factor authentication is implemented for Admin and Caretaker accounts. After entering email and password, users receive a six-digit one-time verification code through Gmail SMTP. The code is valid for 10 minutes.

Passwords are stored using PBKDF2 hashing. Controllers are protected with

[Authorize(Roles = "...")] attributes.

6.2 Role-Based Dashboards

Admin Dashboard:

Displays statistics such as total users, caretakers, orders, menus, and recent system logs. Tables support filtering and pagination.

Caretaker Dashboard:

Displays total orders, completed orders, total revenue, and average ratings. Caretakers can manage menus, ingredients, customizations, and agreements.

User Dashboard:

Displays order statistics, spending information, recent orders, receipt downloads, and rating options.

6.3 Menu System

Caretakers define menu name, description, price, category, image, location, and minimum order quantity.

Ingredient Management:

Menus can include removable ingredients.

Customization System:

A dynamic database-driven customization system was developed. Caretakers can define customization groups and selectable options.

Minimum Order Quantity:

Minimum quantities are enforced according to category.

6.4 Location-Based Filtering

Google Maps JavaScript API and HTML5 Geolocation API are used for location-based filtering. Distances are calculated using the Haversine formula.

6.5 Cart and Payment System

ASP.NET Core Session is used for cart management. Dynamic pricing includes customization costs.

The payment system is a simulation and does not process real payments.

6.6 Email System

MailKit and Gmail SMTP are used for sending order confirmation and notification emails.

6.7 Dynamic PDF Generation

QuestPDF is used to generate:

- Order receipt PDFs
- Caretaker agreement PDFs

6.8 Rating System

Users can rate completed orders with star ratings and comments.

6.9 System Logging

Critical events such as authentication attempts, orders, payments, emails, ratings, menu operations, admin actions, and system errors are logged.

6.10 Live Video Meeting

Jitsi Meet External API enables live meetings between customers and caretakers.

6.11 AI Chatbot

Google Gemini API (gemini-2.0-flash) powers an AI assistant specialized for the Tastemam platform.

6.12 Security

HTTP security headers, OWASP ZAP security scanning, Subresource Integrity (SRI), and .NET User Secrets were implemented to improve security.

7. SEED DATA AND SAMPLE DATA

The application automatically creates:

- Roles: Admin, Caretaker, User
- Sample users
- Caretaker accounts for 25 districts of Ankara
- 125 sample menu items
- Ingredient lists and category-based seed data

8. FILTERING AND PAGINATION

All large tables support filtering and pagination including:

- Admin log table
- Admin orders table
- Admin ratings table
- Caretaker menu table
- User orders table

9. SECURITY ANALYSIS REPORT

A detailed security analysis report was prepared separately under the name:

tastemam_security_report_15052026.pdf

The report explains OWASP ZAP findings, risk evaluations, and implemented security solutions.

10. PROJECT STRUCTURE

LAB1_CENG382/

```

├── Ceng382_Week1/
│   ├── web_project/
│   │   ├── Controllers/
│   │   ├── Data/
│   │   ├── Models/
│   │   ├── Services/
│   │   ├── Views/
│   │   ├── wwwroot/
│   │   └── Program.cs
│   ├── Database/
│   │   └── tastemam_database.sql
│   └── references.txt

```

11. REFERENCES

ASP.NET Core MVC Documentation:

<https://docs.microsoft.com/aspnet/core/mvc>

Entity Framework Core:

<https://docs.microsoft.com/ef/core>

ASP.NET Core Identity:

<https://docs.microsoft.com/aspnet/core/security/authentication/identity>

Google Maps JavaScript API:

<https://developers.google.com/maps/documentation/javascript>

Jitsi Meet External API:

<https://jitsi.github.io/handbook/docs/dev-guide/dev-guide-ljm-api>

Google Gemini API:

<https://ai.google.dev/docs>

QuestPDF:

<https://www.questpdf.com/documentation>

MailKit:

<https://github.com/jstedfast/MailKit>

OWASP ZAP:

<https://www.zaproxy.org>

Bootstrap 5:

<https://getbootstrap.com/docs/5.3>

Font Awesome:

<https://fontawesome.com>

X.PagedList:

<https://github.com/dncuug/X.PagedList>

GitHub Repository

https://github.com/atasoyturk/Ceng382_25-26_202211063

Demo Video

<https://youtu.be/LvmNb23qAFQ>